최기환

Frontend Engineer

rlghks3004@gmail.com GitHub gihwan-dev.tistory.com

통합 인스턴스 모니터링 대시보드와 exem-table 라이브러리, MaxGauge 레거시 5개 제품의 프론트엔드 개발과 유지보수를 혼자 맡고 있습니다. 차세대 MaxGauge에서도 화면 개발과 공통 컴포넌트 개선을 병행하고 있습니다.

01 **소프트웨어가 계속해서 나아질 수 있는 환경을 설계합니다.** 새로운 제품뿐만 아니라 기존 레거시 제품에도 테스트 코드를 도입하고 운영합니다. 코드 수정이 당장의 버그 해결에 그치지 않고 실질적인 개선으로 이어졌는지 확인할 수 있도록 합니다.

02 **좋은 개발자는 회사에 돈을 잘 벌어주는 개발자라고 생각합니다.** 항상 내가 하는 업무의 단기적, 장기적 가치와 필요한 리소스를 저울질하며 올바른 선택을 하려고 노력합니다.

03 **귀찮은 일을 가장 싫어합니다.** 반복되는 작업과 불필요한 일을 최대한 줄이려고 노력합니다.

경력

주식회사엑셈 / 프론트엔드 개발자

2024.11 - 현재

MaxGauge Dashboard: 통합 인스턴스 모니터링

입사 초기부터 담당

`React / TypeScript / TanStack Query`

여러 DB 인스턴스의 상태와 알람을 한 화면에서 확인하는 제품입니다. 입사 초기부터 기존 Vue Dashboard 6.0의 기획과 디자인, 개발을 전담했고, 현재 React 기반의 MaxGauge Dashboard를 맡고 있습니다.

- **고객사별 커스텀 개발을 공통 기능과 기능 플래그로 전환**
  - 고객사별 커스텀 기능이 한 코드에 쌓여, 한 기능을 수정할 때마다 다른 고객사 기능에 미치는 영향까지 확인해야 했습니다.
  - 요청받은 기능을 공통 제품에 추가하고, 해당 고객사의 설정에서 기능 플래그를 켜서 제공하도록 바꿨습니다.
  - 개발과 빌드 시 고객사를 선택하면 그 고객사에 지정한 기능 설정이 적용되도록 도구를 만들었습니다.
- **사내 디자인 시스템을 활용하도록 Vue에서 React로 이전**
  - 별도 기획자와 디자이너 없이 화면을 만들면서 작업마다 디자인과 테마가 달라졌습니다.
  - 기존 코드는 전역 변수에 강하게 의존했고, 그 데이터를 공급하는 API의 전면 변경도 예정돼 있었습니다.
  - 앞으로의 유지보수를 고려해 데이터 구조를 정리하면서 사내 React 디자인 시스템을 도입하기로 했습니다.
  - 대시보드를 React로 이전하고 공통 컴포넌트와 디자인 토큰을 기준으로 화면을 구현했습니다.
- **조회 데이터를 중복 저장하던 전역 상태 제거**
  - React 대시보드는 조회 결과를 전역 스토어에 다시 저장했고, 이를 동기화하는 effect가 불필요한 리렌더링을 일으켰습니다.
  - 위젯이 필요한 서비스 데이터를 직접 조회하고 알람 수를 계산하도록 바꿨습니다.
  - 알람과 인스턴스 전역 스토어를 제거해 조회 결과와 전역 상태를 따로 맞추던 코드를 없앴습니다.
- **모니터링 대상이 바뀌어도 사용자가 정한 위젯 배치 유지**
  - 모니터링 대상이 추가되거나 삭제될 때 저장된 배치를 합치는 코드가 기존 위젯의 위치까지 바꿀 수 있었습니다.
  - 새 대상의 위젯은 빈 공간에 추가하고, 삭제된 대상의 위젯만 제거하도록 바꿨습니다.
  - 대상 추가와 삭제 후에도 나머지 위젯의 위치가 유지되는지를 회귀 테스트로 검증했습니다.

MaxGauge 차세대: DB 성능 분석

`React / TypeScript / TanStack Query / ECharts`

SQL과 세션, DB 객체의 정보를 분석하는 제품입니다. 초기에는 대시보드 위젯 빌더와 SQL Detail, Object Detail 개발을 맡았습니다. 현재는 제품 전반의 기능 수정과 공통 컴포넌트 개발에 참여하고 있습니다.

- **새 시각화를 플러그인으로 추가할 수 있는 위젯 빌더 설계**
  - 새 차트를 추가할 때마다 여러 곳에 흩어진 설정 폼과 검증 규칙, 미리보기 코드를 함께 수정해야 했습니다.
  - 시각화별로 설정과 검증, 미리보기를 하나의 플러그인에 모아 관련 코드를 함께 수정할 수 있도록 했습니다.
  - 새 시각화도 같은 인터페이스로 구현해 등록하면 위젯 빌더에서 설정하고 미리 볼 수 있도록 바꿨습니다.
- **공통 테이블 라이브러리를 적용해 22개 화면의 동작 통일**
  - 화면마다 테이블을 따로 구현해 정렬과 필터 동작, 컬럼 이름과 단위가 제각각이었습니다.
  - 공통 동작은 exem-table 라이브러리가 처리하고, 제품별 컬럼과 상세 화면 이동 규칙은 어댑터에서 관리하도록 나눴습니다.
  - 라이브러리 수정과 22개 화면 전환을 직접 맡아, 각 화면이 컬럼과 데이터만 전달해 같은 테이블 동작을 사용하도록 했습니다.
  - 259개 컬럼 정의를 한곳에 모아 이름과 타입을 화면마다 수정할 필요를 없앴습니다.
- **선택한 SQL이 있는 페이지부터 조회해 대기 시간 단축**
  - 차트에서 고른 SQL이 목록 뒤쪽에 있으면 앞 페이지를 모두 받은 뒤에야 해당 행을 표시했습니다.
  - 선택한 SQL이 있는 페이지부터 조회하고, 앞뒤 데이터는 스크롤할 때 받도록 바꿨습니다.
  - 공통 테이블 라이브러리에 특정 행으로 이동하는 기능을 추가해 선택한 SQL을 바로 찾아가도록 했습니다.
  - 실데이터 20,212건의 한 사례에서 요청은 178건에서 2건으로, 선택 행 표시 시간은 212.5초에서 11.8초로 줄었습니다. (당시 전후 각 1회 비교)
- **성능 측정 후 차트 선명도를 해치던 리사이즈 최적화 제거**
  - 리사이즈 중 캔버스를 늘리고 200ms 뒤 다시 그리는 최적화 때문에 글자와 점이 찌그러졌습니다.
  - CPU 4배 감속 조건에서 21만 점 Scatter의 끊김을 확인했지만, 출시 전부터 드문 대용량 상황에 맞춰 일반적인 화면의 선명도를 포기할 필요는 없다고 판단했습니다.
  - 캔버스 확대와 지연 처리를 제거하고 requestAnimationFrame으로 프레임당 한 번 실제 크기에 맞춰 다시 그리도록 바꿨습니다.
- **반복되던 화면 규약 리뷰를 AI 스킬로 전환**
  - 화면 구성과 공통 컴포넌트 사용 규칙을 리뷰에서 반복해서 확인하고 있었습니다.
  - 규칙과 참조 구현을 스킬로 만들어 AI가 화면을 개발할 때 같은 기준을 따르도록 했습니다.
  - 스킬 도입 후 해당 규약에 대한 리뷰 지적은 더 이상 나오지 않았습니다.
- **변경 전후의 동작을 자동 녹화해 영상으로 리뷰**
  - 스크롤과 드래그처럼 움직임이 중요한 변경은 스크린샷만으로 차이를 판단하기 어려웠습니다.
  - 작업 시나리오를 입력하면 변경 전후에 같은 조작을 녹화하고 비교 영상을 만드는 스킬을 작성했습니다.
  - 리뷰어가 직접 재현하지 않고도 영상에서 동작 차이를 확인할 수 있게 했습니다.

MaxGauge 차세대 (계속)

- **공유 서버를 기다리지 않고 제품을 검증할 수 있는 환경 구성**
  - 공유 테스트 서버는 데이터가 부족하거나 다른 팀이 함께 사용해 필요한 빌드를 올리고 부하를 주기 어려웠습니다.
  - 백엔드와 DB, 수집 데몬의 구성부터 기동과 로그인 확인까지 스크립트와 스킬로 만들었습니다.
  - 공유 서버에 배포하지 않고 원하는 빌드를 검증할 수 있게 했습니다. 원격 Linux에서 기동과 로그인, DB 접속을 확인했습니다.
- **러너 증설 전에 실행 비용을 분석해 CI 시간 단축**
  - 테스트 증가로 CI가 느려졌지만 큐 대기 중앙값은 1.6초였습니다. 러너 분리보다 시간이 드는 실행 단계부터 살펴봤습니다.
  - Storybook은 미리 빌드한 파일로 제공하고, DOM이 필요 없는 테스트는 jsdom 대신 Node 환경에서 실행하도록 바꿨습니다.
  - 보안 검사는 MR에 유지하면서 정적 분석과 리포트는 통합 브랜치로 옮겨 중복 실행을 줄였습니다.
  - 당시 파이프라인 시간 중앙값은 198초에서 94초로 줄었습니다. (변경 전 20회, 후 4회 비교, 재시도와 캐시 재생성 제외)

exem-table: 사내 React 테이블 라이브러리 `2025.06경 인수 - 현재`

`React / TypeScript / TanStack Table / Vitest`

MaxGauge 차세대와 통합 대시보드에서 사용하는 React 테이블 라이브러리입니다. 차세대 프로젝트 안에 있던 컴포넌트를 인수해 API와 렌더링 구조를 개선했고, 현재 개발과 유지보수를 맡고 있습니다.

- **기능별 API와 제어 방식, 비제어 방식의 상태 관리 정리**
  - 사용 화면과 테이블 컴포넌트, TanStack Table이 상태를 중복 관리해 동기화 코드가 복잡했습니다.
  - 정렬과 필터 등 기능별로 관련 옵션을 묶고, 사용하는 화면이 상태를 관리할지 라이브러리에 맡길지 선택하도록 API를 설계했습니다.
  - 테이블 컴포넌트의 중간 상태를 제거해, 제어 방식에서는 사용하는 화면이, 비제어 방식에서는 TanStack Table이 상태를 관리하도록 바꿨습니다.
- **가상화를 기본으로 적용해 두 가지 렌더링 경로를 관리하는 부담 제거**
  - 기존 table 태그 기반 구현에서는 컬럼 리사이즈나 이동 UI를 추가할 때 DOM 배치에 제약이 있었습니다.
  - div 기반으로 바꾸면서 가상화를 기본 동작으로 정했습니다. 가상화 여부에 따라 같은 기능을 두 번 구현하고 검증하는 부담이 더 크다고 판단했습니다.
  - 컬럼 고정과 행 확장도 가상화된 하나의 렌더링 구조에서 처리하도록 했습니다.
- **내부 구조를 바꿔도 기능을 검증할 수 있도록 테스트 재작성**
  - 렌더링 구조를 바꾸자 내부 구현에 의존하던 기존 테스트도 깨졌습니다.
  - 공개 API로 테이블을 렌더링하고 사용자 조작에 따른 화면 결과를 검증하도록 통합 테스트를 다시 작성했습니다.
  - 실제 브라우저에서 정렬과 검색, 선택, 리사이즈를 검증하는 테스트를 쌓아 v9 전환과 상태 관리 구조 변경에도 활용했습니다.
- **TanStack Table v9 전환과 구독 범위 축소로 렌더링 비용 절감**
  - 최상위 컴포넌트가 전체 상태를 구독해 값 하나가 바뀌어도 테이블 전반이 다시 렌더링됐습니다.
  - TanStack 의존 코드를 별도 모듈로 모아 버전 변경 시 수정 범위를 좁혔습니다.
  - v9로 이전하면서 각 컴포넌트가 필요한 상태만 구독하도록 바꿨습니다.
  - v9 전환 후 10,000행, 8컬럼에서 14자 검색어 입력의 누적 React 렌더 시간이 1,927ms에서 61ms로 줄었습니다. (당시 동일 조건 비교, 5회 중앙값)
  - 차세대의 테이블 사용 코드도 새 API로 이전하면서 타입 검사로 잡히지 않던 커스텀 정렬 연결을 4개 화면에서 수정했습니다.

MaxGauge 레거시 제품군 2026.09부터 전담

JavaScript / ExtJS / Playwright / Claude Code / Codex

MFO, MFT, MFA, MFD, MFS는 DB 성능을 모니터링하는 레거시 제품입니다. 2026년 9월부터 5개 제품의 프론트엔드 유지보수를 혼자 맡고 있습니다. 고객 요청을 분석하고 문제를 재현하는 일부터 구현과 검증, 버전 관리와 패키징까지 담당하고 있습니다.

- **실제 화면을 기준으로 설계와 구현 결과를 검토하는 AI 스킬 제작**
  - 기획 문서와 API 명세를 따로 읽으면 화면의 어느 부분을 어떻게 구현할지 파악하기 어려웠습니다.
  - 화면 캡처의 각 영역에 사용할 API와 동작 규칙을 연결해, 구현 방법과 미정 사항을 함께 검토하는 스킬을 만들었습니다.
  - 구현 후에도 실제 화면에 처리 규칙과 선택 이유를 표시해, 세부 코드를 읽지 않고 동작을 확인하고 수정할 부분을 짚을 수 있게 했습니다.
- **레거시에 테스트를 도입해 수정할 때마다 회귀 검증**
  - MFO에 테스트 코드를 도입하고, 수정할 때마다 기존 기능이 깨지지 않는지 확인하고 있습니다.
  - 테스트를 작성하고 검증하는 과정에서 기존 결함 70건을 발견했습니다.
  - 발견한 결함은 다음 릴리즈에서 재현 테스트부터 작성하는 TDD 방식으로 수정할 계획입니다.
- **과거에 반복된 결함을 AI 코드 리뷰의 점검 기준으로 반영**
  - 과거 MR에서 반복된 버그와 사이드 이펙트를 조사해 AI 리뷰 스킬의 점검 항목으로 만들었습니다.
  - 변경한 코드가 리뷰를 받았는지 도구에서 검사하고, 현재 커밋에 대한 리뷰 기록이 없으면 MR 작성과 패키지 전달을 막도록 했습니다.
- **시나리오 이미지로 변경 결과를 확인하는 리뷰 방식 도입**
  - 수정한 기능의 시나리오별 검증 이미지를 MR에 첨부하도록 했습니다.
  - AI 코드 리뷰와 테스트를 먼저 거친 뒤, 이미지에서 의도한 동작을 확인하고 병합하는 흐름으로 바꿨습니다.
  - 리뷰어가 매번 코드를 세세히 읽거나 직접 재현하지 않고도 변경 결과를 확인할 수 있게 했습니다.
- **버전 관리와 배포 절차 정비, 브랜치 운영 규칙 재설계**
  - 어떤 버전에 어떤 작업이 들어갔는지 추적하기 어려웠고, 클라이언트와 SQL의 소스 조합을 잘못 골라 배포본을 만드는 실수도 있었습니다.
  - 버전별 작업 목록과 소스 커밋, 태그를 연결해 패키지에 포함된 변경을 추적할 수 있도록 했습니다.
  - 이미 사용한 버전 번호를 선택하거나 검토 후 배포 대상 소스가 바뀌면 진행을 막도록 했습니다.
  - 업무와 배포 방식에 맞춰 공통 개발, 릴리스, 고객사 브랜치의 역할과 수정 반영 규칙을 다시 정했고, 기존 브랜치의 전환을 진행하고 있습니다.

Task Analyzer: 작업 맥락 검색 도구

실험 프로젝트

`Python / TypeScript / MCP`

개발에 필요한 요구사항과 관련 코드, 과거 작업 이력을 찾는 실험 프로젝트입니다. 검색 기능을 여러 AI 도구에서 재사용할 수 있도록 설계했습니다.

- **여러 AI 도구에서 재사용할 수 있는 검색 서버 설계**
  - Task Analyzer 외의 도구에서도 같은 작업 맥락을 찾을 수 있도록 데이터 관계와 검색 기능을 별도 서버로 분리했습니다.
  - 서버는 근거 검색을 맡고 답변은 호출한 AI가 만들도록 나눠, 특정 AI 도구에 검색 기능이 묶이지 않게 했습니다.
  - MCP와 스킬에서도 같은 검색 API를 쓸 수 있도록 입력과 결과 형식을 정했습니다.
- **검색 플러그인을 배포해 회의록 정리와 개발에 사용**
  - Claude Code와 Codex에서 검색 플러그인을 설치하도록 EXEM AI Marketplace를 구성했습니다.
  - 회의록을 정리할 때 관련 태스크를 찾아 댓글에 링크를 남기고, 개발 중에는 관련 이슈와 변경 이력을 확인하는 데 쓰고 있습니다.

GitLab 슬래시 커맨드: 스레드에서 AI와 협업

실험 프로젝트

`TypeScript / LangGraph`

GitLab MR 댓글에서 AI에 코드 질문이나 수정 요청을 보내고 답변을 함께 검토하는 봇입니다.

- **논의 중인 MR 스레드에서 AI 호출과 결과 공유**
  - 작업 맥락을 개인 PC로 옮기고 AI의 답변을 다시 공유하는 번거로움을 줄이려고 만들었습니다.
  - MR 댓글에서 AI를 호출하면 답변과 수정 결과를 같은 스레드에 남겨 동료도 함께 검토할 수 있게 했습니다.
- **스레드별 대화는 분리하고 작업 결과는 공유**
  - 한 MR 안의 서로 다른 논의가 섞이지 않도록 대화 이력은 스레드별로 나눴습니다.
  - 다른 스레드에서도 앞선 작업을 참고할 수 있도록 결과 요약은 MR 단위로 공유했습니다.
  - GitLab에서 두 스레드의 대화는 분리되고 결과 요약은 다음 대화로 전달되는 동작을 확인했습니다.

학력

동의대학교	정보통신공학과 / 응용 소프트웨어 공학 부전공 | 2017.03 - 2024.03	

최근 글 전체 글 최신 5개 / 2026.10.25 확인

- **자본주의 1부 - 돈은 빚이다**
  2026.09.20
- **지금 테스트 코드가 더더욱 중요한 이유**
  2026.09.06
- **스틱 후기**
  2026.08.29
- **Tanstack Table V9 마이그레이션 회고**
  2026.08.26
- **개발자로 살아남기 - 성장에 대한 고민**
  2026.08.09